deep se
dependable evolvable pervasive software engineering group

Lab session: C functions and Type Conversions

Danilo Ardagna, Roberto Sala

Politecnico di Milano
danilo.ardagna@polimi.it

POLITECNICO DI MILANO

PART 1: C FUNCTIONS

The problem - Sudoku

- In the Sudoku game, a 9x9 grid must be filled with numbers from 1 to 9
- The grid is split in nine rows, nine columns and nine 3x3 subregions; all rows, columns and subregions must contain exactly all numbers from 1 to 9

Example:

```
1 2 3 | 4 5 6 | 7 8 9
4 5 6 | 7 8 9 | 1 2 3
7 8 9 | 1 2 3 | 4 5 6
-------------------------
2 3 4 | 5 6 7 | 8 9 1
5 6 7 | 8 9 1 | 2 3 4
8 9 1 | 2 3 4 | 5 6 7
-------------------------
3 4 5 | 6 7 8 | 9 1 2
6 7 8 | 9 1 2 | 3 4 5
9 1 2 | 3 4 5 | 6 7 8
```

The problem - Sudoku

- **Goal:** write a program that checks whether a given Sudoku matrix is correct

- A search function

```
int search_key (const unsigned v[], unsigned n_elements,
                unsigned key);
```

that returns 1 if `key` is contained in the array `v`, 0 otherwise, is given

- To generate a Sudoku matrix, the given function

```
void generate_sudoku(unsigned sudoku[][SIZE]);
```

can be also used

The problem - Sudoku

You have to implement:

1) a function

```
int basic_search (const unsigned v[], unsigned n_elements);
```

that returns 1 if all numbers from 1 to 9 are contained in v, 0 otherwise

2) three functions

```
int check_rows (const unsigned sudoku[][SIZE]);
int check_cols (const unsigned sudoku[][SIZE]);
int check_regions (const unsigned sudoku[][SIZE]);
```

that return 1 if all the rows, columns and subregions (respectively) of the Sudoku matrix comply with the rules, 0 otherwise

The problem - Sudoku

3) a function

```
int check_sudoku(const unsigned sudoku[][SIZE]);
```

that returns:

- 1 if the given Sudoku matrix complies to all Sudoku rules
- -1 if a row violates the game rules
- -2 if a column violates the game rules
- -3 if a region violates the game rules

Credits

- Lewis' algorithm to generate a Sudoku matrix

https://en.wikipedia.org/wiki/Sudoku_solving_algorithms

PART 2: TYPE CONVERSIONS

Exercise 1

This simple program should, first, apply a temperature correction to a given measurement, and then print a countdown from 5 to 0. **Identify and correct the errors** in this piece of code:

```
int main() {
        unsigned measurement = 10;
        int correction = -20;

        unsigned corrected = measurement + correction;

        std::cout << "Corrected measurement: " << corrected <<
'\n';

        for (unsigned i = 5; i >= 0; --i)
                std::cout << i << ' ';


        return 0;
}
```

Exercise 2

The program should store the squares of numbers from 0 to 9 and print them. **Identify and correct the errors** in this piece of code:

```
int main() {
        std::vector<int> squares;
        for (int i = 0; i < 10; ++i)
                squares[i] = i * i;

        std::cout << "The vector contains " << squares.size() << "
elements\n";

        for (std::size_t i = 0; i <= squares.size(); ++i)
                std::cout << squares[i] << '\n’;

        return 0;
}
```

Exercise 3

The program should add a new delivery to the product's current stock, store the product in a warehouse, and print all products. **Identify and correct the errors** in this piece of code:

```
struct Product {
        std::string name;
        unsigned stock;
};
```

Exercise 3

```
int main()
{
	Product product{"Keyboard", 4000000000};
	product.stock += 1000000000;

	std::vector<Product> warehouse;
	warehouse[0] = product;

	for (std::size_t i = 0; i <= warehouse.size(); ++i)
		std::cout << warehouse[i].name << ": " << warehouse[i].stock << " units\n";

	return 0;
}
```

Yoda's lesson

